# Studio Nx

백엔드 153 — check_reset_rate_limit()

2026-10-14

# 백엔드 153 — `check_reset_rate_limit()` (재설정 요청 횟수 제한)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/backend/app/services/password_reset_service.py` |
| 작업 | P1-28 비밀번호 찾기 (FR-118): 네 번째 함수 |
| 작성 | Claude (AI), 2026-10-14, 사용자 검토 |
| 이전 | 152 `create_reset_token()` |
| 시험 | `tests/test_password_reset_service.py`의 9개 (이 파일 전체 16개) |
| 다음 | `request_password_reset()`: 이메일로 계정을 찾고, 이 함수와 `create_reset_token()`을 부르고, 링크를 서버 로그에 출력 |

## 1. 하는 일

재설정 요청이 너무 잦으면 **429**를 내고, 아니면 이번 요청을 세어 둡니다. 한도는 사용자와 합의한 값입니다.

| 기준 | 한도 (10분 안) | 이유 |
|---|---|---|
| 같은 이메일 | 3번 | 한 사람에게 메일이 쏟아지는 것을 막습니다 |
| 같은 IP | 10번 | 같은 망을 쓰는 여러 사람을 감안해 넉넉하게 둡니다 |

한도를 넘으면 `AppError(TOO_MANY_REQUESTS, 429, "Too many requests. Please try again later.")`이고, `details.retry_after`에 몇 초 뒤에 다시 할 수 있는지를 담습니다.

## 2. 코드 (요약)

```
RESET_LIMIT_SECONDS = 10 * 60
RESET_LIMIT_EMAIL = 3
RESET_LIMIT_IP = 10
_recent_requests: dict[str, deque[float]] = {}

def check_reset_rate_limit(ip: str | None, email: str) -> None:
    now = _clock()
    keys = [(f"email:{email.strip().lower()}", RESET_LIMIT_EMAIL)]
    if ip:
        keys.append((f"ip:{ip}", RESET_LIMIT_IP))
    with _lock:
        retry_after = 0
        for key, limit in keys:
            q = _recent_requests.get(key)
            if q is None:
                continue
            while q and now - q[0] > RESET_LIMIT_SECONDS:
                q.popleft()                     # 10분이 지난 기록은 버림
            if not q:
                del _recent_requests[key]       # 비면 열쇠째 지움
                continue
            if len(q) >= limit:
                retry_after = max(retry_after, int(RESET_LIMIT_SECONDS - (now - q[0])) + 1)
```

```
        if retry_after:
            raise AppError(ErrorCode.TOO_MANY_REQUESTS, 429,
                           "Too many requests. Please try again later.",
                           details={"retry_after": retry_after})
        for key, _ in keys:
            _recent_requests.setdefault(key, deque()).append(now)
```

## 3. 왜 이렇게

| 선택 | 이유 |
|---|---|
| **가입 여부와 관계없이 모든 요청을 셈** | 없는 이메일만 제한에 안 걸리거나 먼저 걸리면 응답 차이로 가입 여부를 알아낼 수 있습니다. FR-118의 "항상 같은 응답"을 지키기 위해서입니다 |
| 문의와 달리 **검사와 세기를 한 함수로** | 문의는 저장에 실패한 시도를 세지 않으려고 둘로 나눴습니다. 여기서는 가입 여부를 숨겨야 해서 항상 세므로 나눌 이유가 없습니다 |
| **한도를 넘은 요청은 세지 않음** | 계속 눌러도 제한 시간이 늘어나지 않고, `retry_after`가 시간이 지나며 줄어듭니다 |
| 이메일은 소문자와 공백 제거 | `A@Example.com`과 `a@example.com`이 다른 사람으로 세어지면 제한을 피할 수 있습니다 |
| 둘 중 하나라도 넘으면 막음 | 이메일 기준과 IP 기준을 모두 검사하고, `retry_after`는 둘 중 긴 쪽입니다. 막히지 않았을 때만 둘을 함께 셉니다 |
| 메모리 방식 | 문의(FR-113)와 같은 결정입니다. 서버를 다시 켜면 비워지고, 서버가 여러 대면 따로 셉니다. 정식 운영에서 서버를 늘리면 Redis 같은 곳으로 옮겨야 합니다 |
| 오래된 기록 정리 | 열쇠가 비면 지워서, 여러 이메일로 요청해도 메모리가 계속 늘지 않습니다 |
| 시계를 `_clock`으로 분리 | 시험에서 시간을 직접 움직입니다(`time.monotonic`: 시스템 시계를 바꿔도 영향이 없음) |

## 4. 시험 (9개 추가, 파일 전체 16개 통과)

| 확인 | 결과 |
|---|---|
| 같은 이메일은 3번 통과하고 4번째에 429 | ✓ |
| 이메일은 대소문자와 앞뒤 공백을 무시 | ✓ |
| 같은 IP는 이메일이 달라도 10번까지, 11번째에 429 | ✓ |
| 다른 이메일과 다른 IP는 영향 없음 | ✓ |
| 막힌 요청은 세지 않고, `retry_after`가 시간이 지나며 줄어듦 | ✓ |
| 10분이 지나면 다시 통과 | ✓ |
| **가입된 이메일과 없는 이메일이 똑같이 3번 통과하고 4번째에 막힘** | ✓ |
| IP가 없으면 이메일 기준만 적용 | ✓ |
| 만료된 기록이 정리되어 메모리가 늘지 않음 | ✓ |

**시험을 확인하는 시험.** 일부러 틀리게 바꿔 시험이 잡는지 보았습니다.

- 한도 비교를 `>=`에서 `>`로 바꾸자 6개가 실패했습니다.
- 막힌 요청도 세도록 바꾸자 9개가 실패했습니다.
- 둘 다 원래대로 되돌린 뒤 통과했습니다.

전체 pytest 717개 통과, `ruff` 통과.